SERVA

Technical Specification

Universal Booking & Payments Marketplace

Version 1.0 | April 2026

Table of Contents

- 1. Executive Summary
- 2. Platform Architecture
- 3. Complete Database Schema
- 4. Complete API Endpoints
- 5. Authentication & Authorization
- 6. Real-Time Booking Engine
- 7. Payment Architecture
- 8. Search & Discovery System
- 9. Notification System
- 10. Form Builder Architecture
- 11. Consumer App Screens
- 12. Business Dashboard Screens
- 13. Admin Panel Screens
- 14. Environment & Configuration

1. Executive Summary

Serva is a universal booking and payments marketplace that enables consumers to discover, book, and pay for any appointment-based service. The platform consists of three applications: a consumer mobile app, a business dashboard, and an admin panel.

MVP Tech Stack

- Frontend: React Native + Expo (mobile), Next.js + TypeScript (web)
- Backend: Node.js with Supabase (PostgreSQL database)
- Authentication: Supabase Auth with JWT
- Storage: Supabase Storage (images, documents)
- Payments: Stripe Connect (platform accounts, ACH payouts)
- Hosting: Vercel (frontend), Supabase (backend)
- Email: Resend (transactional emails)
- SMS: Twilio (SMS notifications)
- Push Notifications: Expo Notifications (mobile)

Three Applications

- Consumer Mobile App: Search, discover, book, and pay for services
- Business Dashboard: Manage profile, services, calendar, bookings, and intake forms
- Admin Panel: Moderate businesses, approve services, view analytics

2. Platform Architecture

Serva uses a microservices-inspired architecture with clear separation of concerns:

- API Layer: RESTful endpoints built with Next.js API routes
- Database Layer: PostgreSQL via Supabase with real-time subscriptions
- Authentication: Supabase Auth with JWT tokens and RLS policies
- Storage: Supabase Storage for images, files, and documents
- Payment Processing: Stripe Connect for platform payments and payouts
- Real-time: Supabase Realtime for booking updates, notifications
- Email: Resend for transactional and marketing emails
- SMS: Twilio for SMS notifications and reminders
- Push: Expo Notifications for mobile push notifications

Data Flow

- Consumer searches for services using search & discovery
- Available time slots are calculated from availability rules
- Booking is created with temporary slot lock to prevent double-booking
- Payment intent is created with Stripe
- Payment is authorized and immediately captured
- Booking confirmation is sent via email, SMS, and push
- Business receives booking notification with customer intake data
- Payout is scheduled for business bank account

3. Complete Database Schema

The Serva database uses PostgreSQL with Supabase. Below is the complete schema with all tables, columns, constraints, and relationships.

3.1 Users

Core user records authenticated via Supabase Auth.

Column	Type & Constraints
id	UUID PRIMARY KEY
auth_id	UUID (Supabase Auth user ID)
email	TEXT UNIQUE NOT NULL
phone	TEXT UNIQUE
first_name	TEXT
last_name	TEXT
role	ENUM (consumer, business, admin)
profile_photo_url	TEXT
created_at	TIMESTAMPTZ DEFAULT NOW()
updated_at	TIMESTAMPTZ DEFAULT NOW()
deleted_at	TIMESTAMPTZ (soft delete)

Indexes: (email), (auth_id), (role), (deleted_at)

RLS Policy: Users can read own record, admins can read all

3.2 User Profiles

Extended user profile information for consumers.

Column	Type & Constraints
id	UUID PRIMARY KEY
user_id	UUID FK users(id) NOT NULL
bio	TEXT
date_of_birth	DATE
address	TEXT
city	TEXT

state_province	TEXT
postal_code	TEXT
country	TEXT
timezone	TEXT
preferred_language	TEXT DEFAULT 'en'
created_at	TIMESTAMPTZ DEFAULT NOW()
updated_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (user_id)

3.3 Businesses

Business accounts that provide services.

Column	Type & Constraints
id	UUID PRIMARY KEY
owner_id	UUID FK users(id) NOT NULL
name	TEXT NOT NULL
description	TEXT
phone	TEXT
email	TEXT
website	TEXT
address	TEXT
city	TEXT
state_province	TEXT
postal_code	TEXT
country	TEXT
location	GEOGRAPHY(POINT,4326) (lat/long)
category_id	UUID FK categories(id)
subcategory_id	UUID FK subcategories(id)
rating	DECIMAL(3,2) DEFAULT 0
review_count	INTEGER DEFAULT 0
stripe_account_id	TEXT UNIQUE

status	ENUM (pending, approved, suspended)
profile_photo_url	TEXT
banner_photo_url	TEXT
verification_date	TIMESTAMPTZ
created_at	TIMESTAMPTZ DEFAULT NOW()
updated_at	TIMESTAMPTZ DEFAULT NOW()
deleted_at	TIMESTAMPTZ (soft delete)

Indexes: (owner_id), (category_id), (location), (status), (stripe_account_id)

RLS: Owners can read/update own, consumers can read approved only

3.4 Business Photos

Gallery photos for business listings.

Column	Type & Constraints
id	UUID PRIMARY KEY
business_id	UUID FK businesses(id) NOT NULL
photo_url	TEXT NOT NULL
alt_text	TEXT
display_order	INTEGER
created_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (business_id, display_order)

3.5 Categories

Service categories (e.g., Health & Wellness, Beauty, Professional Services).

Column	Type & Constraints
id	UUID PRIMARY KEY
name	TEXT UNIQUE NOT NULL
slug	TEXT UNIQUE NOT NULL
description	TEXT
icon_url	TEXT
display_order	INTEGER

created_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (slug)

3.6 Subcategories

Service subcategories (e.g., Haircut, Massage, Consulting).

Column	Type & Constraints
id	UUID PRIMARY KEY
category_id	UUID FK categories(id) NOT NULL
name	TEXT NOT NULL
slug	TEXT NOT NULL
description	TEXT
icon_url	TEXT
display_order	INTEGER
created_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (category_id, slug), (slug)

3.7 Services

Individual services offered by a business.

Column	Type & Constraints
id	UUID PRIMARY KEY
business_id	UUID FK businesses(id) NOT NULL
name	TEXT NOT NULL
description	TEXT
duration_minutes	INTEGER NOT NULL
price	DECIMAL(10,2) NOT NULL
deposit_amount	DECIMAL(10,2) DEFAULT 0
cancellation_fee_percent	INTEGER DEFAULT 0
buffer_before_minutes	INTEGER DEFAULT 0
buffer_after_minutes	INTEGER DEFAULT 0
max_attendees	INTEGER DEFAULT 1

active	BOOLEAN DEFAULT true
color	TEXT (hex color for calendar)
display_order	INTEGER
created_at	TIMESTAMPTZ DEFAULT NOW()
updated_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (business_id, active), (business_id)

3.8 Staff

Staff members of a business.

Column	Type & Constraints
id	UUID PRIMARY KEY
business_id	UUID FK businesses(id) NOT NULL
name	TEXT NOT NULL
email	TEXT
phone	TEXT
photo_url	TEXT
bio	TEXT
role	TEXT
active	BOOLEAN DEFAULT true
created_at	TIMESTAMPTZ DEFAULT NOW()
updated_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (business_id, active)

3.9 Staff Services

Maps which staff can provide which services (many-to-many).

Column	Type & Constraints
id	UUID PRIMARY KEY
staff_id	UUID FK staff(id) NOT NULL
service_id	UUID FK services(id) NOT NULL
created_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (staff_id, service_id), (service_id)

Unique constraint: (staff_id, service_id)

3.10 Availability Rules

Define recurring availability patterns (e.g., Monday-Friday 9am-5pm).

Column	Type & Constraints
id	UUID PRIMARY KEY
business_id	UUID FK businesses(id) NOT NULL
staff_id	UUID FK staff(id)
day_of_week	INTEGER (0=Sun, 1=Mon, ..., 6=Sat)
start_time	TIME NOT NULL
end_time	TIME NOT NULL
active	BOOLEAN DEFAULT true
created_at	TIMESTAMPTZ DEFAULT NOW()
updated_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (business_id, day_of_week), (staff_id, day_of_week)

3.11 Availability Overrides

Exception dates (closed, special hours, etc.).

Column	Type & Constraints
id	UUID PRIMARY KEY
business_id	UUID FK businesses(id) NOT NULL
staff_id	UUID FK staff(id)
date	DATE NOT NULL
start_time	TIME
end_time	TIME
is_blocked	BOOLEAN DEFAULT true (true=unavailable)
reason	TEXT (e.g., holiday, appointment)
created_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (business_id, date), (staff_id, date)

3.12 Bookings

Core booking records.

Column	Type & Constraints
id	UUID PRIMARY KEY
consumer_id	UUID FK users(id) NOT NULL
business_id	UUID FK businesses(id) NOT NULL
service_id	UUID FK services(id) NOT NULL
staff_id	UUID FK staff(id)
start_datetime	TIMESTAMPTZ NOT NULL
end_datetime	TIMESTAMPTZ NOT NULL
slot_lock_id	TEXT (unique slot lock key)
slot_lock_expires	TIMESTAMPTZ
status	ENUM (pending, confirmed, completed, cancelled)
cancellation_reason	TEXT
cancelled_by	ENUM (consumer, business, admin)
cancelled_at	TIMESTAMPTZ
num_attendees	INTEGER DEFAULT 1
notes	TEXT (consumer notes)
form_submission_id	UUID FK form_submissions(id)
created_at	TIMESTAMPTZ DEFAULT NOW()
updated_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (consumer_id, status), (business_id, start_datetime), (service_id), (slot_lock_id)

RLS: Consumers see own, businesses see own business, admins see all

3.13 Booking Status History

Audit trail of booking status changes.

Column	Type & Constraints
id	UUID PRIMARY KEY

booking_id	UUID FK bookings(id) NOT NULL
old_status	ENUM
new_status	ENUM
changed_by	UUID FK users(id)
reason	TEXT
created_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (booking_id, created_at)

3.14 Payments

Payment records with Stripe details.

Column	Type & Constraints
id	UUID PRIMARY KEY
booking_id	UUID FK bookings(id) NOT NULL
stripe_payment_intent_id	TEXT UNIQUE
stripe_charge_id	TEXT
amount_cents	BIGINT NOT NULL (in cents)
amount_earned_cents	BIGINT (platform share)
amount_payout_cents	BIGINT (business share)
currency	TEXT DEFAULT 'USD'
status	ENUM (pending, authorized, captured, failed)
payment_method	TEXT (card, bank_account)
last_4	TEXT (last 4 digits)
error_message	TEXT
created_at	TIMESTAMPTZ DEFAULT NOW()
updated_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (booking_id), (stripe_payment_intent_id), (status)

3.15 Refunds

Refund records linked to payments.

Column	Type & Constraints

id	UUID PRIMARY KEY
payment_id	UUID FK payments(id) NOT NULL
stripe_refund_id	TEXT UNIQUE
amount_cents	BIGINT NOT NULL
reason	ENUM (cancellation, customer_request, admin)
status	ENUM (pending, completed, failed)
created_by	UUID FK users(id)
created_at	TIMESTAMPTZ DEFAULT NOW()
updated_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (payment_id), (stripe_refund_id)

3.16 Payouts

Track business payouts to connected Stripe accounts.

Column	Type & Constraints
id	UUID PRIMARY KEY
business_id	UUID FK businesses(id) NOT NULL
stripe_payout_id	TEXT UNIQUE
amount_cents	BIGINT NOT NULL
currency	TEXT DEFAULT 'USD'
status	ENUM (pending, in_transit, paid, failed, cancelled)
period_start	DATE
period_end	DATE
created_at	TIMESTAMPTZ DEFAULT NOW()
updated_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (business_id, status), (stripe_payout_id)

3.17 Reviews

Customer reviews and ratings.

Column	Type & Constraints

id	UUID PRIMARY KEY
booking_id	UUID FK bookings(id) NOT NULL UNIQUE
business_id	UUID FK businesses(id) NOT NULL
consumer_id	UUID FK users(id) NOT NULL
rating	INTEGER (1-5) NOT NULL
title	TEXT
comment	TEXT
is_verified_booking	BOOLEAN DEFAULT true
helpful_count	INTEGER DEFAULT 0
status	ENUM (pending, approved, rejected)
created_at	TIMESTAMPTZ DEFAULT NOW()
updated_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (business_id, rating), (consumer_id), (status)

3.18 Review Responses

Business responses to reviews.

Column	Type & Constraints
id	UUID PRIMARY KEY
review_id	UUID FK reviews(id) NOT NULL
business_id	UUID FK businesses(id) NOT NULL
response_text	TEXT NOT NULL
created_at	TIMESTAMPTZ DEFAULT NOW()
updated_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (review_id), (business_id)

3.19 Form Templates

Intake form templates created by businesses.

Column	Type & Constraints
id	UUID PRIMARY KEY
business_id	UUID FK businesses(id) NOT NULL

service_id	UUID FK services(id)
name	TEXT NOT NULL
description	TEXT
required_before_booking	BOOLEAN DEFAULT false
active	BOOLEAN DEFAULT true
created_at	TIMESTAMPTZ DEFAULT NOW()
updated_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (business_id, service_id)

3.20 Form Fields

Individual fields within form templates.

Column	Type & Constraints
id	UUID PRIMARY KEY
form_template_id	UUID FK form_templates(id) NOT NULL
name	TEXT NOT NULL
type	ENUM (text, email, phone, date, checkbox, select, textarea)
label	TEXT NOT NULL
placeholder	TEXT
required	BOOLEAN DEFAULT false
options	JSONB (for select type)
validation_pattern	TEXT (regex)
conditional_logic	JSONB (show if...)
display_order	INTEGER
created_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (form_template_id, display_order)

3.21 Form Submissions

Submitted form responses.

Column	Type & Constraints
id	UUID PRIMARY KEY

form_template_id	UUID FK form_templates(id) NOT NULL
booking_id	UUID FK bookings(id)
consumer_id	UUID FK users(id) NOT NULL
submitted_data	JSONB (key-value pairs)
created_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (booking_id), (form_template_id, created_at)

3.22 Notifications

Notification records (email, SMS, push).

Column	Type & Constraints
id	UUID PRIMARY KEY
user_id	UUID FK users(id) NOT NULL
type	ENUM (email, sms, push)
event	ENUM (booking_confirmed, booking_reminder, etc.)
recipient	TEXT (email or phone)
subject	TEXT
body	TEXT
related_id	UUID (booking_id or other)
status	ENUM (queued, sent, failed, bounced)
sent_at	TIMESTAMPTZ
error_message	TEXT
created_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (user_id, status), (created_at), (type, status)

3.23 Notification Preferences

User notification settings.

Column	Type & Constraints
id	UUID PRIMARY KEY
user_id	UUID FK users(id) NOT NULL
booking_confirmed_email	BOOLEAN DEFAULT true

booking_confirmed_sms	BOOLEAN DEFAULT true
booking_confirmed_push	BOOLEAN DEFAULT true
booking_reminder_email	BOOLEAN DEFAULT true
booking_reminder_sms	BOOLEAN DEFAULT false
booking_reminder_push	BOOLEAN DEFAULT true
cancellation_email	BOOLEAN DEFAULT true
review_request_email	BOOLEAN DEFAULT true
marketing_email	BOOLEAN DEFAULT false
updated_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (user_id)

3.24 Favorites

Consumer saved/favorited businesses.

Column	Type & Constraints
id	UUID PRIMARY KEY
consumer_id	UUID FK users(id) NOT NULL
business_id	UUID FK businesses(id) NOT NULL
created_at	TIMESTAMPTZ DEFAULT NOW()

Indexes: (consumer_id, business_id), (created_at DESC for sorting)

Unique constraint: (consumer_id, business_id)

3.25 Indexes Summary

Key performance indexes for common queries:

- bookings: (consumer_id, status), (business_id, start_datetime), (slot_lock_id)
- businesses: (owner_id), (category_id), (location for geo queries), (status)
- availability_rules: (business_id, day_of_week), (staff_id)
- payments: (booking_id), (stripe_payment_intent_id), (status)
- reviews: (business_id, rating), (status) for moderation
- form_submissions: (booking_id), (form_template_id, created_at)
- notifications: (user_id, status), (created_at)

3.26 Row Level Security Policies

Supabase RLS policies enforce data access:

- Users: Can read own record, admins can read all
- Bookings: Consumers see own, businesses see own business bookings, admins see all
- Businesses: Anyone can read approved, owners can read/update own, admins can read all
- Services: Only admins/business owners can create, anyone can read active
- Reviews: Anyone can read approved, admins moderate pending
- Form Templates: Businesses can manage own, consumers can only view/submit

4. Complete API Endpoints

All API endpoints are built with Next.js API routes and follow RESTful conventions. Authentication uses Supabase Auth JWT tokens passed in the Authorization header.

4.1 Authentication Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register new user (consumer/business)
POST	/api/auth/login	Login with email/password
POST	/api/auth/logout	Logout (client-side token removal)
POST	/api/auth/refresh	Refresh JWT token
POST	/api/auth/forgot-password	Request password reset link
POST	/api/auth/reset-password	Reset password with token
POST	/api/auth/verify-email	Verify email address

4.2 User Endpoints

Method	Endpoint	Description
GET	/api/users/me	Get current user profile
PUT	/api/users/me	Update current user (auth required)
GET	/api/users/:id	Get public user profile
PUT	/api/users/:id/profile	Update user profile (auth required)
POST	/api/users/avatar	Upload avatar (auth required)
DELETE	/api/users/me	Delete account (auth required)

4.3 Business Endpoints

Method	Endpoint	Description
GET	/api/businesses	List businesses (filters, pagination)

POST	/api/businesses	Create business (auth required, business role)
GET	/api/businesses/:id	Get business details
PUT	/api/businesses/:id	Update business (owner/admin)
DELETE	/api/businesses/:id	Delete business (owner/admin)
POST	/api/businesses/:id/photos	Upload business photos (owner)
GET	/api/businesses/:id/photos	Get business photos
DELETE	/api/businesses/:id/ photos/:photoId	Delete photo (owner)
POST	/api/businesses/:id/reviews	Get business reviews

4.4 Service Endpoints

Method	Endpoint	Description
GET	/api/businesses/:businessId/ services	List services
POST	/api/businesses/:businessId/ services	Create service (owner)
PUT	/api/services/:id	Update service (owner)
DELETE	/api/services/:id	Delete service (owner)

4.5 Staff Endpoints

Method	Endpoint	Description
GET	/api/businesses/:businessId/staff	List staff (owner)
POST	/api/businesses/:businessId/staff	Add staff member (owner)
PUT	/api/staff/:id	Update staff (owner)
DELETE	/api/staff/:id	Delete staff (owner)
POST	/api/staff/:id/services	Assign service to staff (owner)

4.6 Availability Endpoints

Method	Endpoint	Description
GET	/api/availability/slots	Get available slots for service/date

POST	/api/availability/rules	Create availability rule (owner)
GET	/api/availability/rules	List rules (owner)
PUT	/api/availability/rules/:id	Update rule (owner)
DELETE	/api/availability/rules/:id	Delete rule (owner)
POST	/api/availability/overrides	Create override (owner)
GET	/api/availability/overrides	List overrides (owner)
DELETE	/api/availability/overrides/:id	Delete override (owner)

4.7 Booking Endpoints

Method	Endpoint	Description
POST	/api/bookings	Create booking (auth required, with lock)
GET	/api/bookings	List bookings (consumer/business)
GET	/api/bookings/:id	Get booking details
PUT	/api/bookings/:id	Update booking
POST	/api/bookings/:id/cancel	Cancel booking
POST	/api/bookings/:id/reschedule	Reschedule booking
POST	/api/bookings/:id/confirm	Confirm booking (business)
POST	/api/bookings/:id/complete	Mark complete (business)

4.8 Payment Endpoints

Method	Endpoint	Description
POST	/api/payments/intent	Create payment intent (after booking)
POST	/api/payments/confirm	Confirm payment (capture charge)
POST	/api/payments/:id/refund	Refund payment (admin/owner)
GET	/api/payments/:bookingId	Get payment by booking (auth)
GET	/api/payouts	List business payouts (owner/admin)

4.9 Review Endpoints

Method	Endpoint	Description
POST	/api/reviews	Submit review (auth required)
GET	/api/businesses/:businessId/reviews	List business reviews
PUT	/api/reviews/:id	Update review (author)
DELETE	/api/reviews/:id	Delete review (author/admin)
POST	/api/reviews/:id/response	Add response (business owner)
PUT	/api/reviews/:id/response	Update response (business owner)

4.10 Form Endpoints

Method	Endpoint	Description
POST	/api/forms	Create form template (owner)
GET	/api/businesses/:businessId/forms	List forms (owner)
PUT	/api/forms/:id	Update form (owner)
DELETE	/api/forms/:id	Delete form (owner)
POST	/api/forms/:id/fields	Add field to form (owner)
POST	/api/forms/:id/submit	Submit form (auth required)
GET	/api/forms/:id/submissions	List submissions (owner)

4.11 Notification Endpoints

Method	Endpoint	Description
GET	/api/notifications	List notifications (auth required)
PUT	/api/notifications/:id/read	Mark as read
PUT	/api/notifications/preferences	Update preferences
GET	/api/notifications/preferences	Get preferences

4.12 Search Endpoints

Method	Endpoint	Description

GET	/api/search	Full-text search (query, filters, geo)
GET	/api/categories	List categories
GET	/api/categories/:id/subcategories	List subcategories

4.13 Admin Endpoints

Method	Endpoint	Description
GET	/api/admin/businesses/pending	List pending businesses (admin)
PUT	/api/admin/businesses/:id/approve	Approve business (admin)
PUT	/api/admin/businesses/:id/reject	Reject business (admin)
PUT	/api/admin/businesses/:id/suspend	Suspend business (admin)
GET	/api/admin/reviews/pending	List pending reviews (admin)
PUT	/api/admin/reviews/:id/approve	Approve review (admin)
DELETE	/api/admin/reviews/:id	Remove review (admin)
GET	/api/admin/analytics	Get platform analytics (admin)
GET	/api/admin/disputes	List disputes (admin)

4.14 Stripe Webhooks

Method	Endpoint	Description
POST	/api/webhooks/stripe	Handle Stripe events (no auth)

Handles: charge.succeeded, charge.failed, charge.refunded, payout.paid, payout.failed

5. Authentication & Authorization

5.1 Supabase Auth Flow

- User registers with email and password
- Supabase generates JWT token with auth_id and user role
- Token is stored in secure HTTP-only cookie (web) or secure storage (mobile)
- Token expires after 1 hour (refresh token valid 7 days)
- All API requests include Authorization: Bearer <token> header
- Backend validates JWT signature and extracts user_id from token

5.2 Role-Based Access Control

- consumer: Can create bookings, pay, leave reviews, manage own profile
- business: Can manage own business, services, staff, bookings, payouts
- admin: Full platform access, moderation, analytics, dispute resolution

5.3 Row Level Security (RLS) Policies

- All tables have RLS enabled in Supabase
- SELECT: Users can read own records, admins can read all
- INSERT: Authenticated users can insert own data (e.g., bookings)
- UPDATE: Users can only update own records (enforced by RLS policy)
- DELETE: Soft deletes only, hard delete for admins only

5.4 RLS Policy Examples

Example: Users can only read their own user record unless admin

```
CREATE POLICY users_select ON users FOR SELECT USING (auth.uid()::text =
id::text OR auth.jwt()->>'role' = 'admin');
```

Example: Businesses can only update their own business

```
CREATE POLICY businesses_update ON businesses FOR UPDATE USING
(owner_id::text = auth.uid()::text);
```

Example: Consumers can only see bookings they own or their business bookings

```
CREATE POLICY bookings_select ON bookings FOR SELECT USING
(consumer_id::text = auth.uid()::text OR business_id IN (SELECT id FROM businesses
WHERE owner_id::text = auth.uid()::text));
```

6. Real-Time Booking Engine

6.1 Availability Slot Calculation

- System fetches availability_rules for business/staff and requested date
- Calculates recurring time windows (e.g., Mon 9am-5pm)
- Subtracts booked slots and buffers (before/after service duration)
- Applies overrides (holidays, special hours, closures)
- Returns array of 15-min or 30-min slots for selection

Algorithm: For each day, iterate through availability windows. For each possible start time, check if service duration + buffers fit without overlapping existing bookings.

6.2 Slot Locking Mechanism

- When consumer selects a slot, a slot_lock is created with 5-min TTL
- Lock uses a unique key: business_id:staff_id:service_id:datetime
- Prevents double-booking during checkout (consumer goes to payment)
- Lock is removed when payment is captured or expires
- Redis could be used for performance, but PostgreSQL + scheduled cleanup works for MVP

6.3 Double-Booking Prevention

- Database constraint: Unique index on (staff_id, start_datetime, end_datetime) for confirmed bookings
- Check if any existing booking overlaps with new booking time
- Booking creation uses database transaction to ensure atomicity
- If lock expires before payment, user must re-select slot

6.4 Cancellation & Refund Flow

- Consumer clicks cancel on booking detail page
- System checks cancellation deadline (typically 24 hours before)
- If within cancellation period: refund = full payment minus cancellation fee
- If outside cancellation period: no refund, 100% cancellation fee applied
- Refund request is sent to Stripe via refund API
- Booking status changes to 'cancelled'
- Business receives cancellation notification
- Consumer receives refund confirmation

7. Payment Architecture

7.1 Stripe Connect Onboarding

- Business clicks 'Connect Stripe Account' in dashboard
- Redirected to Stripe OAuth flow with client_id, redirect_uri
- User completes Stripe verification (bank details, identity)
- Redirected back with code, exchanged for Stripe account ID
- Stripe account ID stored in businesses.stripe_account_id
- Account is in restricted mode until verification completes

7.2 Payment Flow: Intent → Authorize → Capture

- 1. Consumer books and proceeds to payment
- 2. Frontend calls POST /api/payments/intent with booking_id
- 3. Backend creates Stripe PaymentIntent with amount, currency, metadata
- 4. Stripe returns client_secret to frontend
- 5. Frontend uses Stripe.js to mount payment form (Stripe Elements)
- 6. Consumer enters card details (or selects saved payment method)
- 7. Frontend calls Stripe.confirmPayment() to create Payment
- 8. Stripe processes payment and returns payment_intent.succeeded
- 9. Frontend calls POST /api/payments/confirm with payment_intent_id
- 10. Backend retrieves PaymentIntent from Stripe (confirms amount, metadata)
- 11. Backend updates payments table with status='captured'
- 12. Booking status changes to 'confirmed'
- 13. Notifications queued (email, SMS, push)

7.3 Platform Fee Structure

- Service price set by business (e.g., $100)
- Platform fee: 10% (platform cut) or configurable
- Deposit (if required): Separate charge or percentage of service
- Tip: Optional, added at checkout, no platform fee on tip
- Calculation: Payout = (Price - Platform Fee) + Tip - Refund

7.4 Deposit Flow

- Business can require deposits (e.g., 20% of service price)
- At booking, only deposit is charged (not full amount)
- Full payment must be completed 24 hours before appointment
- If not paid, booking auto-cancels and deposit is refunded

7.5 Tip Flow

- Tip option presented at checkout (e.g., 15%, 18%, 20%, custom)
- Tip is charged separately from service price
- Tip amount goes entirely to business (no platform fee)
- Reflected in payout calculation

7.6 Cancellation Fee Flow

- Business sets cancellation fee percentage (0-100%, default 0)
- When booking is cancelled within cancellation window:
- Refund = (Service Price - (Service Price * Cancellation Fee %))
- Fee is kept by platform and business based on split
- Refund is processed back to original payment method via Stripe

7.7 Payout Scheduling

- Payouts are batched daily (e.g., 3 PM ET)
- Funds are held for 3-day rolling reserve (configurable)
- After 3 days, funds are automatically paid out to Stripe Connect account
- Payout method: ACH (2-5 business days) or instant payout (if enabled)
- Payout records are tracked in payouts table with status

7.8 Webhook Handling

- Stripe sends webhooks to /api/webhooks/stripe
- Webhook events: charge.succeeded, charge.failed, charge.refunded, etc.
- Backend verifies webhook signature using Stripe secret
- Updates payment and booking status based on event
- Idempotent: Safe to retry webhooks without duplication

8. Search & Discovery System

8.1 PostgreSQL Full-Text Search

- Businesses have searchable text: name, description, category, service names
- Full-text search uses PostgreSQL tsvector and tsquery
- Query: SELECT * FROM businesses WHERE to_tsvector('english', name || ' ' || description) @@ plainto_tsquery('english', 'yoga massage');
- Ranks results by relevance (ts_rank function)

8.2 PostGIS Geospatial Queries

- Businesses have location column: GEOGRAPHY(POINT,4326) (lat, long)
- Geo search: Find all businesses within X miles of consumer location
- Query: SELECT * FROM businesses WHERE ST_DWithin(location, ST_MakePoint(lon, lat), 5000) ORDER BY ST_Distance(location, ST_MakePoint(lon, lat));
- Supports distance calculations in meters/miles

8.3 Search Ranking Algorithm

- Relevance score = full_text_rank * 0.4 + rating_score * 0.3 + distance_score * 0.3
- Full-text rank: PostgreSQL ts_rank (0-1)
- Rating score: (business_rating / 5) * 100 (0-100)
- Distance score: 100 - (distance_km / max_distance_km * 100) (biased toward nearby)
- Results filtered by category/subcategory, availability, price range

8.4 Search Filters

- Category: Select category and subcategory
- Price Range: Min and max price per service
- Rating: Minimum rating (1-5 stars)
- Distance: Max distance from consumer location (miles)
- Availability: Only show available slots for selected date/time
- Favorites: Filter to only consumer's saved businesses

8.5 Discovery Features

- Home Page: Featured businesses (admin-curated or algorithm-driven)
- Browse by Category: Grid of categories, tap to see subcategories
- Trending: Most booked services this week (based on booking count)
- Nearby: Sorted by distance from consumer location
- Top Rated: Sorted by rating and review count
- New Businesses: Recently joined marketplace

9. Notification System

9.1 Notification Types

- Email: Transactional (via Resend), marketing (bulk)
- SMS: Text messages via Twilio
- Push: Mobile push notifications via Expo Notifications

9.2 Notification Triggers

- booking_confirmed: Sent 2 minutes after payment is captured
- booking_reminder: Sent 24 hours before appointment, 1 hour before
- booking_cancelled: Sent when booking is cancelled
- review_request: Sent 24 hours after booking completion
- business_new_booking: Sent to business when new booking is created
- payment_failed: Sent if payment fails during checkout
- refund_processed: Sent when refund is issued

9.3 Email Templates (via Resend)

- booking_confirmed: Service details, location, date/time, cancellation policy
- booking_reminder: Appointment reminder with link to booking
- booking_cancelled: Refund status, refund amount
- review_request: Link to leave review, incentive text
- password_reset: Reset link with 1-hour expiry
- email_verification: Verification link
- new_booking_business: Customer name, service, time, intake form data

9.4 SMS Templates (via Twilio)

- booking_confirmed: 'Your appointment is confirmed on [DATE] at [TIME]. Reply CONFIRM to confirm or CANCEL to cancel.'
- booking_reminder: 'Reminder: Your [SERVICE] with [BUSINESS] is in 1 hour at [TIME]. [CANCELLATION POLICY]'
- verification: 'Your Serva verification code is [CODE]. Valid for 10 minutes.'

9.5 Push Notifications (via Expo)

- Platform: iOS and Android via Expo Notifications
- Device tokens stored in user_notification_tokens table
- Triggered for: booking_confirmed, booking_reminder, business new booking, promotions
- Deep link: Open booking detail or business profile from notification tap

9.6 Notification Preferences

- Users can opt in/out of each notification type (email, SMS, push)
- Preferences stored in notification_preferences table
- Default: Email on, SMS off, Push on
- Unsubscribe link in all transactional emails

9.7 Notification Queuing & Delivery

- Notifications are inserted into notifications table with status='queued'
- Background job runs every 5 minutes to send queued notifications
- Job calls Resend API (email), Twilio API (SMS), Expo API (push)
- Status updated to 'sent', 'failed', or 'bounced'
- Retry failed notifications up to 3 times with exponential backoff

10. Form Builder Architecture

10.1 Data Model

- form_templates: Template definition (name, description, required_before_booking)
- form_fields: Individual fields (type, label, required, options, validation)
- form_submissions: Submitted responses (JSONB key-value pairs)
- form_field_values: Individual field responses (indexed for query)

10.2 Field Types

- text: Single-line text input
- email: Email input with validation
- phone: Phone input with formatting
- date: Date picker
- checkbox: Single or multiple checkboxes
- select: Dropdown menu (options provided in JSONB)
- textarea: Multi-line text input

10.3 Conditional Logic

- Fields can be shown/hidden based on previous answers
- Stored as JSONB: { show_if: { field_id: value } }
- Example: Show 'Dietary restrictions' only if 'Dietary concerns?' = true

10.4 Validation

- Client-side: HTML5 validation, regex patterns
- Server-side: Validate each field against type and pattern
- Required fields checked before form submission allowed

10.5 Consumer Experience

- Form is shown after service selection, before payment
- Responses are stored in booking.form_submission_id
- Business can view responses in booking detail
- Responses exported as PDF or CSV

11. Consumer App Screens (React Native/Expo)

~20 core screens across tab navigation and modal stacks

11.1 Tab Navigation

- Home: Feed, featured, nearby, trending
- Search: Search bar, filters, results, map view
- Bookings: Upcoming, past, cancelled bookings
- Favorites: Saved businesses
- Profile: User profile, settings, bookings, reviews, notifications

11.2 Screen List

1. Home: Featured carousel, nearby services, trending, trending categories
2. Search: Full-text search, filters (category, price, distance), map view
3. Search Results: Grid/list, sorting (relevance, distance, rating, price)
4. Business Profile: Photos, services, staff, hours, reviews, about, map
5. Service Detail: Service info, price, duration, deposit, cancellation policy, reviews
6. Date/Time Selection: Calendar picker, time slot picker, availability
7. Attendees: Select number of attendees (if applicable)
8. Intake Form: Dynamic form with conditional logic
9. Payment: Card entry, saved cards, tip selection, total breakdown
10. Booking Confirmation: Confirmation number, details, receipt, add to calendar
11. Booking Detail: Active or past booking details, reschedule/cancel options
12. Bookings List: Upcoming tab, past tab, cancelled tab, search/filter
13. Favorites: Saved businesses, remove from favorites, sort
14. Leave Review: Star rating, title, comment, photo upload
15. View Reviews: List of reviews, helpful votes, business responses
16. User Profile: Name, email, phone, profile photo, bio, addresses
17. Edit Profile: Update personal info, change password, delete account
18. Payment Methods: Add/remove cards, set default
19. Notifications: List of notifications, mark as read, clear
20. Settings: Notification preferences, language, timezone, account

11.3 Navigation Structure

- Bottom Tab Navigator: Home, Search, Bookings, Favorites, Profile
- Stack Navigator within each tab for drill-down
- Modal Stack for Auth (Login, Register, Reset Password)
- Modal Stack for Booking (Service Detail → Intake → Payment → Confirmation)

11.4 Authentication Flow

- Login/Register: Email/password, social login (Google, Apple)
- Forgot Password: Email input, reset link sent, new password entry
- Token stored securely: AsyncStorage (mobile), HttpOnly cookie (web)
- Refresh token automatically when JWT expires

12. Business Dashboard Screens (Next.js)

~18 core screens for business management

12.1 Main Navigation

- Sidebar: Dashboard, Profile, Services, Staff, Calendar, Bookings, Forms, Reviews, Analytics, Stripe Setup, Settings

12.2 Screen List

21. Dashboard: Key metrics (total revenue, bookings this week, average rating, pending approvals)
22. Profile Editor: Business info, address, phone, email, website, hours, photos, category
23. Services: List of services, create/edit/delete, pricing, duration, buffers, capacity
24. Staff: Add/manage staff, assign services, enable/disable
25. Availability Rules: Set recurring hours (Mon-Fri 9am-5pm), override dates, holidays
26. Calendar: Month/week/day view, booked slots, click to see booking details
27. Bookings: List view, filter by status, search, bulk actions (confirm, complete, cancel)
28. Booking Detail: Customer info, service, time, form responses, confirmation/completion/cancellation
29. Forms: List of intake form templates, create/edit/delete, field management
30. Form Builder: Add fields, set types, required, conditional logic, preview
31. Form Submissions: View responses, export to CSV/PDF
32. Reviews: List reviews, pending, approved, add responses, moderation tools
33. Analytics: Revenue chart, bookings trend, customer acquisition, top services, ratings over time
34. Stripe Setup: Connect account status, verification status, linked account details, payout history
35. Payouts: List payouts, status, amount, date, export history
36. Settings: Business name, tax ID, cancellation policy, deposit requirements, notification preferences
37. Account: Owner info, change password, manage staff permissions, delete account

12.3 Navigation Structure

- Sidebar navigation on desktop, collapsible on tablet/mobile
- Breadcrumb navigation for sub-pages
- Modal dialogs for create/edit operations

12.4 Key Features

- Real-time updates: Calendar updates via Supabase Realtime when new bookings arrive
- Bulk actions: Multi-select bookings for batch confirmation/cancellation
- Export: Bookings, reviews, analytics exported as CSV/PDF

- Webhooks: Receive POST requests for new bookings (optional integration)

13. Admin Panel Screens (Next.js)

~8 core screens for platform administration

13.1 Main Navigation

- Sidebar: Dashboard, Business Queue, Users, Reviews, Categories, Analytics, Settings, Disputes

13.2 Screen List

38. Dashboard: Platform health (total revenue, active businesses, user count, support tickets)
39. Business Queue: Pending business approvals, details, approve/reject/suspend
40. Businesses: Search/filter, view details, suspend/unsuspend, analytics per business
41. Users: Search users, view activity, suspend/delete (soft), export
42. Reviews: Pending review moderation, approve/reject/remove, view context
43. Disputes: Handle disputes between consumer and business, refund decisions
44. Categories: Manage categories and subcategories, CRUD operations
45. Analytics: Platform metrics (revenue, user acquisition, top services, geographic distribution)
46. Settings: Platform fees, deposit requirements, payment terms, email templates, feature flags

13.3 Moderation Tools

- Review Queue: Flag/approve/reject reviews before publishing
- Dispute Resolution: View both sides, make payout decisions, send notifications
- Bulk Actions: Suspend multiple users/businesses, ban keywords

14. Environment & Configuration

14.1 Environment Variables

- SUPABASE_URL: Supabase project URL
- SUPABASE_ANON_KEY: Public anon key for client-side auth
- SUPABASE_SERVICE_ROLE_KEY: Service role for backend operations
- STRIPE_SECRET_KEY: Stripe secret key for payment processing
- STRIPE_PUBLIC_KEY: Stripe public key for frontend Stripe.js
- STRIPE_WEBHOOK_SECRET: Webhook signing secret
- RESEND_API_KEY: Resend API key for email
- TWILIO_ACCOUNT_SID: Twilio account SID
- TWILIO_AUTH_TOKEN: Twilio auth token
- TWILIO_PHONE_NUMBER: Twilio phone number for SMS
- EXPO_PUSH_TOKEN_KEY: Expo push notification key
- NEXT_PUBLIC_APP_URL: Frontend URL for redirects
- JWT_SECRET: Secret for additional JWT signing (if used)
- DATABASE_URL: Direct PostgreSQL connection string (backend only)

14.2 Feature Flags

- FEATURE_DEPOSITS: Enable/disable deposit requirements
- FEATURE_INSTANT_PAYOUTS: Enable/disable instant payout option
- FEATURE_INTAKE_FORMS: Enable/disable form builder
- FEATURE_BULK_BOOKING: Enable/disable multi-attendee bookings
- PLATFORM_FEE_PERCENT: Percentage fee taken by platform (0-20%)

14.3 Database Setup Checklist

- Create Supabase project
- Enable Realtime on key tables (bookings, notifications)
- Enable RLS on all tables
- Create all tables from schema above
- Create indexes for performance
- Set up authentication with email/password
- Configure JWT expiry (1 hour) and refresh token (7 days)

14.4 Stripe Setup Checklist

- Create Stripe account
- Activate Stripe Connect (for platform payment splits)
- Create API keys (secret and public)
- Set up webhook endpoint (/api/webhooks/stripe)
- Subscribe to events: charge.succeeded, charge.failed, charge.refunded, payout.paid

- Configure OAuth redirect URL for business onboarding

14.5 Email Setup Checklist (Resend)

- Create Resend account
- Verify sender domain
- Create email templates (confirmation, reminder, receipt, reset)
- Set up API key in environment variables
- Test email delivery

14.6 SMS Setup Checklist (Twilio)

- Create Twilio account
- Purchase phone number (US or international)
- Set up API credentials (Account SID, Auth Token)
- Create message templates
- Set up webhook for SMS delivery status

14.7 Push Notification Setup (Expo)

- Set up Expo account and project
- Configure EAS Build for APK/IPA generation
- Generate push notification token in app
- Store token in database linked to user_id
- Test push delivery via Expo Notifications API

14.8 Hosting Setup

- Deploy Next.js apps to Vercel (backend API)
- Deploy React Native app via Expo (over-the-air updates)
- Database and auth managed by Supabase
- Static assets (photos, documents) stored in Supabase Storage
- CDN: Vercel CDN for frontend, Supabase Storage CDN for assets

14.9 Security Checklist

- Enable HTTPS on all endpoints
- Set CORS headers to allow only known domains
- Validate all input on backend (email, phone, zip codes)
- Sanitize database queries (no SQL injection)
- Rate limit API endpoints (10 requests/sec per IP)
- Hash passwords (Supabase Auth handles this)
- Encrypt sensitive data (Stripe tokens, PII)
- Enable 2FA for admin accounts

- Log all admin actions (moderation, refunds, suspensions)
- Regular security audits and penetration testing

15. Deployment & Operations

15.1 Development Environment

- Local: Next.js dev server, Expo dev client, Supabase local instance (emulator)
- Git: Feature branches, pull requests with code review
- Linting: ESLint, Prettier for code formatting
- Testing: Jest for unit tests, React Testing Library for components

15.2 Staging Environment

- Database: Separate Supabase project (staging)
- Stripe: Test mode (stripe.com/docs/testing)
- Deployment: Vercel preview deployments (auto-deploy from PR)
- QA: Manual testing before production release

15.3 Production Environment

- Database: Supabase production project with automated backups
- Stripe: Live mode
- Deployment: Vercel production (main branch only)
- Monitoring: Sentry for error tracking, LogRocket for session replay
- Uptime: Monitor.com or similar for 99.9% SLA monitoring

15.4 Database Backups

- Automated daily backups via Supabase
- 30-day retention
- Manual backup before major releases
- Test restore procedure quarterly

15.5 Monitoring & Alerts

- Sentry: Alert on errors (P0 immediately, P1 daily digest)
- Datadog: Monitor API latency, database query performance
- PagerDuty: On-call rotation for production incidents
- Status page: Public status at status.serva.com

15.6 Scaling Considerations

- Database: Supabase auto-scales, but may need read replicas at 10k+ users
- Cache: Add Redis cache for availability calculations as load increases
- CDN: Vercel and Supabase Storage CDN handle static assets

- Queue: Consider job queue (BullMQ, Celery) if notification volume exceeds 1k/sec

15.7 Cost Optimization

- Supabase: ~$500/month for production (auto-scaling with usage)
- Vercel: ~$100/month for serverless functions + bandwidth
- Stripe: 2.9% + 30¢ per transaction
- Resend: $0.20 per email (or flat fee)
- Twilio: $0.0075 per SMS (or SMS bundles)
- Expo: Free for small user base, $99/month for EAS Build
- Total: ~$1000/month at launch, scales with revenue

16. Technical Debt & Future Roadmap

16.1 Known Limitations (MVP)

- No multi-location support (one location per business)
- No inventory management (seat capacity only)
- No recurring/subscription bookings (one-off appointments only)
- No calendar sync (Google Calendar, Outlook)
- No automated SMS reminders (manual setup required)
- Limited reporting (basic analytics only)

16.2 Post-MVP Roadmap

- Q3 2026: Multi-location, calendar sync, advanced analytics
- Q4 2026: Subscription bookings, membership plans, loyalty rewards
- 2027: Team collaboration, advanced permissions, API for integrations
- 2027+: AI scheduling, automated marketing, white-label option

17. Appendix: Technical Glossary

- JWT: JSON Web Token for stateless authentication
- RLS: Row Level Security in PostgreSQL for data access control
- tsvector: PostgreSQL text search vector for full-text indexing
- Geography: PostGIS data type for geospatial queries
- Stripe Connect: Platform payment network for business accounts
- PaymentIntent: Stripe object representing a payment transaction
- Webhook: HTTP POST callback for event notification
- Idempotent: Operation that produces same result when repeated
- Slot Lock: Temporary exclusive lock preventing double-booking
- Realtime: Live data synchronization via WebSocket
- JSONB: PostgreSQL JSON data type for flexible schema
- DXA: Document pixel unit (1/20th of a point, used in Word)

18. Document Information

- Version: 1.0
- Last Updated: April 2026
- Author: Serva Engineering
- Status: Final
- Next Review: Q3 2026